

INSTITUTO TECNOLÓGICO UNIVERSITARIO

Universidad Nacional de Cuyo — ITU
Espacio Curricular: Gestión de Infraestructura (EGI)

ECOSISTEMA DE INVENTARIO
SEGURO

Documentación Técnica del Proyecto Integrador

Equipo de Desarrollo — Comisión B

Integrante	Rol	Rama Git
Lorenzo Zamparini	DevOps	main / feature/devops-k8s
Kevin Nieto	Lider /Documentacion	main
Máximo Muñoz	Base de Datos SQL Server	feature/sql-db
Juan Manuel Atencio	Base de Datos MongoDB	feature/mongo-db
Francisco Navas	LDAP + Red / Network Policies	feature/ldap-network
Pablo Penoff	Aplicación Flask	feature/flask-app

Índice de Contenidos

1. Introducción y Contexto del Problema

1.1 Descripción del Problema

El Instituto Tecnológico Universitario (ITU–UNCUYO) necesita un sistema centralizado para inventariar las computadoras de sus laboratorios de informática. La gestión manual resulta ineficiente y propicia a errores, impidiendo conocer en tiempo real el estado, ubicación y componentes de cada equipo.

El área de infraestructura institucional exige que el despliegue sea altamente seguro, cumpliendo con las políticas de red perimetrales del Firewall universitario (GUFW o pfSense) y que tanto los usuarios de las bases de datos como de la aplicación se autenticuen contra el servidor Active Directory/LDAP institucional.

1.2 Alcance del Sistema

La aplicación web debe ser capaz de:

- Consultar SQL Server para obtener la ubicación de cada equipo (aula, laboratorio, número de banco/mesa, fecha de mantenimiento) y su responsable (técnico, alumno o docente asignado temporalmente).
- Usar el identificador único del equipo para obtener de MongoDB sus componentes internos: fabricante, modelo, CPU, RAM, disco, tipo (desktop/laptop/workstation), sistema operativo y periféricos.
- Gestionar asignaciones de equipos, solicitudes de uso y registro de mantenimientos.
- Autenticar a todos los usuarios contra el servidor Active Directory/LDAP institucional.
- Desplegar todos los servicios de forma contenerizada sobre Kubernetes (Minikube) con políticas de red Zero-Trust.

1.3 Objetivos del Proyecto

1. Implementar una arquitectura de microservicios con separación clara de responsabilidades entre el frontend Flask, la base relacional SQL Server y la base documental MongoDB.
2. Configurar políticas de red Kubernetes (NetworkPolicies) que implementen el principio de menor privilegio.
3. Integrar autenticación LDAP/Active Directory con soporte de fallback local para entornos de desarrollo.
4. Demostrar operación de MongoDB desde shell: inserciones, consultas filtradas, actualizaciones y eliminaciones.
5. Versionar todo el código y los manifiestos en un repositorio Git con ramas por funcionalidad.

2. Arquitectura del Sistema

2.1 Visión General

El ecosistema está compuesto por cinco servicios principales que corren dentro del namespace inventario-seguro en Minikube. El tráfico externo ingresa a través de un NodePort que simula el NAT del pfSense/GUFW, pasando obligatoriamente por la IP de la DMZ del Firewall. Internamente, las NetworkPolicies de Calico implementan Zero-Trust: todo tráfico está bloqueado por defecto y solo se permiten los flujos estrictamente necesarios.

2.2 Componentes del Ecosistema

Servicio	Nombre en K8s	Imagen	Puerto	Tipo de Servicio
Frontend Web (Flask)	inventario-web	inventario-web:latest	5000	NodePort (30080)
Base MongoDB	inventario-db	mongo:4.4	27017	ClusterIP
Base SQL Server	ubicacion-db	mcr.microsoft.com/mssql/server:2022	1433	ClusterIP
Servidor LDAP/AD	ldap-service	osixia/openldap (externo o mock)	389	ClusterIP
Simulación Firewall	—	GUFW / pfSense (host)	—	Host Network

2.3 Diagrama de Arquitectura de Servicios

A continuación se describe el flujo de comunicación entre los componentes del ecosistema:

Origen	Destino	Puerto	Protocolo	Política
Internet / Usuario	inventario-web	:30080 → :5000	TCP/HTTP	allow-external-to-web
inventario-web	inventario-db	:27017	TCP	allow-web-to-mongo
inventario-web	ubicacion-db	:1433	TCP	allow-web-to-sql (host network)
inventario-web	ldap-service	:389	TCP/LDAP	allow-web-to-ldap
Cualquier otro tráfico	Cualquier destino	—	—	deny-all (bloqueado)
Todos los pods	kube-dns	:53	UDP/TCP	allow-dns

2.4 Configuración del Namespace y Labels

Todos los recursos residen en el namespace inventario-seguro, etiquetado con proyecto: egi-itu y equipo: inventario. El cluster se inicializa con CNI Calico para habilitar el soporte de NetworkPolicies:

```
minikube start --cni=calico

# Verificar que Calico esté activo:
kubectl get pods -n kube-system | grep calico

# Aplicar el namespace:
kubectl apply -f k8s/namespace.yaml
```

2.5 Stack Tecnológico

Capa	Tecnología	Versión	Rol
Frontend	Flask + Jinja2	3.0.0	Aplicación web y lógica de negocio
ORM / DB	pymssql	2.3.13	Conector Python a SQL Server
NoSQL Client	pymongo	4.5.0	Conector Python a MongoDB
Auth	ldap3	2.9.1	Autenticación LDAP / Active Directory
Config	python-dotenv	1.0.0	Variables de entorno
Container	Docker	latest	Empaquetado de la aplicación
Orquestación	Kubernetes (Minikube)	v1.29+	Despliegue y gestión de servicios
CNI	Calico	latest	Network Policies Zero-Trust
DB Relacional	SQL Server 2022 Express	2022	Datos de ubicación y asignación
DB Documental	MongoDB	4.4	Componentes de hardware

3. Modelo de Base de Datos

3.1 Base de Datos Relacional — SQL Server (ubicacion_db)

SQL Server almacena toda la información de ubicación física, asignación y mantenimiento de los equipos. El modelo sigue las formas normales estándar para garantizar integridad referencial.

3.1.1 Entidades y Atributos

Tabla: aulas

Columna	Tipo	Restricciones	Descripción
id_aula	INT IDENTITY(1,1)	PRIMARY KEY	Identificador autoincremental
nombre	VARCHAR(100)	NOT NULL	Nombre del laboratorio o aula
tipo	VARCHAR(20)	CHECK IN ('aula','laboratorio')	Tipo de espacio físico
piso	INT	NULL	Número de piso del edificio
edificio	VARCHAR(50)	NULL	Nombre o código del edificio

Tabla: responsables

Columna	Tipo	Restricciones	Descripción
id_responsable	INT IDENTITY(1,1)	PRIMARY KEY	Identificador autoincremental
nombre	VARCHAR(100)	NOT NULL	Nombre del responsable
apellido	VARCHAR(100)	NOT NULL	Apellido del responsable
email	VARCHAR(150)	UNIQUE NOT NULL	Email institucional

Tabla: equipos (tabla central)

Columna	Tipo	Restricciones	Descripción
id_equipo	VARCHAR(20)	PRIMARY KEY	ID único (ej: PC-LAB1-001)
codigo_inventario	VARCHAR(50)	UNIQUE NOT NULL	Código de inventario institucional
id_aula	INT	FK → aulas, NOT NULL	Aula donde se encuentra
numero_banco	VARCHAR(10)	NOT NULL	Número de banco o mesa
estado	VARCHAR(20)	CHECK IN 5 valores	activo / asignado / en_reparacion / disponible / baja
id_responsable	INT	FK → responsables NULL	Responsable técnico asignado

Columna	Tipo	Restricciones	Descripción
fecha_ingreso	DATE	NOT NULL	Fecha de alta en el sistema

Tabla: asignaciones

Columna	Tipo	Restricciones	Descripción
id_asignacion	INT IDENTITY(1,1)	PRIMARY KEY	Identificador autoincremental
id_equipo	VARCHAR(20)	FK → equipos NOT NULL	Equipo asignado
usuario_uid	VARCHAR(50)	NOT NULL	UID LDAP del usuario asignado
rol_assigned	VARCHAR(20)	CHECK: alumno/docente/tecnico	Rol del usuario al momento de la asignación
fecha_desde	DATE	NOT NULL	Inicio de la asignación
fecha_hasta	DATE	NULL	Fin de la asignación (NULL = indefinida)
activa	BIT	DEFAULT 1	1 = activa, 0 = finalizada

Tabla: mantenimientos

Columna	Tipo	Restricciones	Descripción
id_mant	INT IDENTITY(1,1)	PRIMARY KEY	Identificador autoincremental
id_equipo	VARCHAR(20)	FK → equipos NOT NULL	Equipo mantenido
fecha_mant	DATE	NOT NULL	Fecha del mantenimiento
tipo	VARCHAR(30)	CHECK: 3 valores	preventivo / correctivo / limpieza
descripcion	VARCHAR(500)	NULL	Detalle de las tareas realizadas
tecnico	VARCHAR(100)	NULL	Nombre del técnico actuante
proximo_mant	DATE	NULL	Fecha sugerida del próximo mantenimiento

3.1.2 Relaciones entre Entidades

- aulas → equipos: una aula puede tener muchos equipos (1:N)
- responsables → equipos: un responsable puede tener varios equipos (1:N)
- equipos → asignaciones: un equipo puede tener múltiples asignaciones históricas (1:N)
- equipos → mantenimientos: un equipo puede tener múltiples mantenimientos (1:N)

3.2 Base de Datos Documental — MongoDB (inventario)

MongoDB almacena los datos de hardware de cada equipo. Al ser documental, cada documento puede tener estructura variable, lo que permite registrar equipos heterogéneos sin alterar el esquema.

3.2.1 Colección: hardware

Esquema de documento tipo (desktop/workstation):

```
{
  "id_equipo": "PC-LAB1-001",          // Clave de enlace con SQL Server
  "fabricante": "Dell",
  "modelo": "OptiPlex 3080",
  "tipo": "desktop",                  // desktop | laptop | workstation
  "cpu": {
    "marca": "Intel",
    "modelo": "Core i5-10500",
    "nucleos": 6,
    "frecuencia_ghz": 3.1
  },
  "ram": {
    "cantidad_gb": 16,
    "tipo": "DDR4",
    "frecuencia_mhz": 2666
  },
  "discos": [
    { "tipo": "SSD", "interfaz": "SATA", "capacidad_gb": 480, "marca": "Kingston"
  ]
  },
  "sistema_operativo": {
    "nombre": "Windows 11 Pro",
    "version": "22H2"
  },
  "perifericos": {
    "monitor": "Dell 24 E2422H",
    "teclado": "Dell KB216",
    "mouse": "Dell MS116"
  },
  "fecha_registro": ISODate("2023-03-01")
}
```

3.2.2 Colección: solicitudes

Almacena las solicitudes de equipo realizadas por alumnos y docentes, pendientes de aprobación administrativa.

```
{
  "usuario": "alumno.uid",
  "nombre_completo": "María Gómez",
}
```



```
"rol": "alumno",
"fecha_solicitud": ISODate("2025-05-01T10:00:00Z"),
"motivo": "Proyecto final de carrera",
"fecha_necesidad": "2025-06-15",
"estado": "pendiente"    // pendiente | aprobada | rechazada
}
```

3.2.3 Consultas de Demostración en Shell MongoDB

Las siguientes consultas demuestran las operaciones CRUD requeridas por el enunciado, ejecutables desde el shell del contenedor:

```
# Ingresar al shell del contenedor MongoDB:
kubectl exec -it deployment/inventario-db -n inventario-seguro -- mongo \
  -u root -p <password> --authenticationDatabase admin inventario

# 1. INSERTAR un documento:
db.hardware.insertOne({
  id_equipo: "PC-TEST-001", fabricante: "Asus",
  modelo: "ProArt", tipo: "workstation",
  cpu: { marca: "AMD", modelo: "Ryzen 9 5900X", nucleos: 12, frecuencia_ghz: 3.7
},
  ram: { cantidad_gb: 32, tipo: "DDR4", frecuencia_mhz: 3600 },
  discos: [{ tipo: "SSD", interfaz: "NVMe", capacidad_gb: 1000, marca: "WD" }],
  sistema_operativo: { nombre: "Ubuntu", version: "22.04 LTS" },
  perifericos: { monitor: "Asus ProArt 27", teclado: "Logitech MX", mouse:
"Logitech MX" },
  fecha_registro: new Date()
})

# 2. BUSCAR con filtro (laptops con más de 8 GB de RAM):
db.hardware.find({ tipo: "laptop", "ram.cantidad_gb": { $gt: 8 } })

# 3. ACTUALIZAR un registro (ampliar RAM):
db.hardware.updateOne(
  { id_equipo: "PC-LAB1-003" },
  { $set: { "ram.cantidad_gb": 16 } }
)

# 4. ELIMINAR un documento específico:
db.hardware.deleteOne({ id_equipo: "PC-TEST-001" })

# 5. Contar documentos por tipo:
db.hardware.aggregate([{$group: { _id: "$tipo", total: { $sum: 1 } } }])
```

4. Aplicación Web (inventario-web)

4.1 Descripción General

La aplicación está desarrollada en Python con el framework Flask y Jinja2 para el renderizado de plantillas HTML. Implementa el patrón MVC de forma simplificada: las rutas en `app.py` actúan como controladores, las plantillas Jinja2 como vistas y los módulos `db.py` y `auth.py` como la capa de modelo/datos.

4.2 Estructura de Módulos

Módulo	Responsabilidad
<code>app.py</code>	Punto de entrada. Define todas las rutas/vistas y la lógica de negocio.
<code>auth.py</code>	Autenticación (LDAP/local), permisos por rol, menú lateral dinámico.
<code>db.py</code>	Conexiones a SQL Server (pymssql) y MongoDB (pymongo). Todas las queries.
<code>config.py</code>	Configuración centralizada. Lee variables de entorno con <code>python-dotenv</code> .
<code>requirements.txt</code>	Dependencias Python del proyecto (Flask, pymssql, pymongo, ldap3, python-dotenv).

4.3 Módulo de Autenticación (auth.py)

El sistema soporta dos modos de autenticación configurables mediante la variable de entorno `AUTH_MODE`:

- Modo LDAP (producción): se conecta al Active Directory institucional, realiza un bind con la cuenta de servicio, busca al usuario por `sAMAccountName`, verifica la contraseña con un segundo bind, extrae los grupos del atributo `memberOf` y mapea al rol de la aplicación.
- Modo local (desarrollo): valida contra un diccionario Python en memoria, útil para pruebas sin infraestructura AD disponible.

4.3.1 Roles y Permisos

Rol	Grupo AD	Permisos en la Aplicación
admin	G_Administradores	Dashboard, CRUD completo de equipos, componentes, mantenimiento, asignaciones, aprobar solicitudes
tecnico	G_Tecnicos	Dashboard, ver equipos, CRUD componentes, registrar mantenimiento, ver asignaciones
docente	G_Docentes	Dashboard, ver equipos, ver asignaciones, solicitar equipo
alumno	G_Alumnos	Ver detalle de equipo, ver asignaciones propias, solicitar equipo

4.4 Módulo de Base de Datos (db.py)

Implementa el patrón de conexión bajo demanda: cada operación abre su propia conexión y la cierra en el bloque finally. Para MongoDB se usa un cliente singleton para reutilizar el pool de conexiones.

4.4.1 Consulta Políglota (Cross-Database)

La vista /equipo/<id> realiza una consulta cruzada entre ambas bases de datos:

```
# Vista equipo_detalle en app.py
@app.route('/equipo/<id>')
def equipo_detalle(id):
    # 1. Consultar SQL Server: ubicación y responsable
    sql_details = get_equipo_sql_details(id)
    # 2. Consultar MongoDB: componentes de hardware
    mongo_details = get_hardware_by_id(id)
    # 3. Renderizar vista unificada
    return render_template('equipo_detalle.html',
        sql_details=sql_details, mongo_details=mongo_details)
```

4.5 Rutas y Funcionalidades

Ruta	Método	Permiso	Descripción
/	GET	—	Redirecciona a /login
/login	GET/POST	—	Formulario de inicio de sesión
/logout	GET	—	Cierre de sesión
/equipos	GET	dashboard	Dashboard con estadísticas y tabla de equipos
/equipo/<id>	GET	equipo_detalle	Detalle unificado SQL+MongoDB
/equipo/nuevo	GET/POST	equipo_nuevo	Formulario para agregar equipo en ambas BDs
/componentes	GET	componentes	Lista de hardware en MongoDB con filtro
/componentes/nuevo	GET/POST	componentes	Alta de componentes en MongoDB
/componentes/<id>/editar	GET/POST	componentes	Edición de componentes en MongoDB
/componentes/<id>/eliminar	POST	componentes	Eliminación de componentes en MongoDB
/mantenimiento	GET	mantenimiento	Lista de registros de mantenimiento
/mantenimiento/nuevo	GET/POST	mantenimiento	Registro de nuevo mantenimiento en SQL Server
/asignaciones	GET	asignaciones	Lista de asignaciones activas (filtrada por rol)

Ruta	Método	Permiso	Descripción
/solicitar-equipos	GET/POST	solicitar_equipos	Formulario para solicitar un equipo
/asignaciones/aprobar/<id>	POST	aprobar_solicitudes	Aprobación de solicitud por admin
/asignaciones/rechazar/<id>	POST	aprobar_solicitudes	Rechazo de solicitud por admin
/health	GET	—	Health check: estado de SQL y MongoDB

4.6 Flujograma de la Aplicación

A continuación se describe el flujo principal de operación del sistema, desde el inicio de sesión hasta las operaciones CRUD:

Etapas	Descripción del Flujo
1. Inicio de Sesión	El usuario accede a /login → ingresa credenciales → el sistema intenta autenticación LDAP; si AUTH_MODE=local usa el diccionario mock → si es exitosa crea sesión y redirige al Dashboard.
2. Dashboard	El sistema consulta SQL Server: total de equipos, laboratorios, asignados, próximos mantenimientos y distribución por aula (gráfico circular conic-gradient).
3. Ver Detalle de Equipo	El usuario selecciona un equipo → /equipo/<id> → consulta paralela a SQL Server (ubicación, responsable) y MongoDB (componentes) → vista unificada.
4. Alta de Equipo	Admin completa formulario → se validan campos obligatorios → se inserta primero en SQL Server (id_equipo, código_inventario, aula, banco) → luego en MongoDB (hardware) → flash de éxito o error parcial.
5. Solicitar Equipo	Alumno/Docente accede a /solicitar-equipos → completa motivo y fecha → se guarda en colección MongoDB solicitudes con estado=pendiente.
6. Aprobar Solicitud	Admin ve solicitudes pendientes en /asignaciones → selecciona equipo disponible y fecha de fin → se crea registro en tabla asignaciones de SQL Server → se actualiza estado en MongoDB.
7. Registrar Mantenimiento	Técnico/Admin accede a /mantenimiento/nuevo → selecciona equipo, tipo, fecha, técnico → se inserta en tabla mantenimientos de SQL Server → el estado del equipo se actualiza a 'en_reparacion'.
8. Cierre de Sesión	El usuario accede a /logout → se limpia la sesión Flask → redirección a /login.

5. Despliegue en Kubernetes (Minikube)

5.1 Estructura de Manifiestos

Archivo	Tipo de Recurso	Descripción
k8s/namespace.yaml	Namespace	Crea el namespace inventario-seguro
k8s/secrets.yaml	Secret	Credenciales cifradas (SQL SA, Mongo, LDAP bind, Flask secret)
k8s/frontend/configmap.yaml	ConfigMap	Variables no sensibles: IPs, puertos, dominios AD
k8s/frontend/deployment.yaml	Deployment+Service	App Flask: 1 réplica, hostNetwork, NodePort 30080
k8s/frontend/secret.yaml	Secret	Secret específico del frontend
k8s/mongo-db/deployment.yaml	Deployment+Service	MongoDB 4.4: ClusterIP, PVC persistente
k8s/mongo-db/pvc.yaml	PersistentVolumeClaim	Almacenamiento persistente para /data/db
k8s/mongo-db/seed-job.yaml	Job	Carga inicial de 10 documentos de hardware
k8s/sql-db/deployment.yaml	Deployment+Service	SQL Server 2022: ClusterIP, PVC persistente
k8s/sql-db/pvc.yaml	PersistentVolumeClaim	Almacenamiento persistente para /var/opt/mssql
k8s/sql-server/sql-seed-job.yaml	Job+ConfigMap	Crea esquema relacional y datos iniciales vía sqlcmd
k8s/network-policies/01-deny-all.yaml	NetworkPolicy	Bloquea todo el tráfico Ingress y Egress por defecto
k8s/network-policies/02-allow-external-to-web.yaml	NetworkPolicy	Permite tráfico externo al frontend en :5000
k8s/network-policies/03-allow-web-to-mongo.yaml	NetworkPolicy	Permite que el frontend acceda a MongoDB en :27017
k8s/network-policies/04-allow-dns.yaml	NetworkPolicy	Permite resolución DNS para todos los pods

5.2 Políticas de Red Zero-Trust

Las NetworkPolicies implementan el principio de menor privilegio. La política base deniega todo y las reglas específicas solo habilitan los flujos necesarios:

5.2.1 deny-all (base)

Se aplica a todos los pods del namespace mediante un podSelector vacío. Bloquea tanto Ingress como Egress. Es el punto de partida del modelo Zero-Trust.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: inventario-seguro
spec:
  podSelector: {}    # Aplica a TODOS los pods
  policyTypes:
    - Ingress
    - Egress
```

5.2.2 allow-external-to-web

Permite que cualquier origen (internet, simulación pfSense) pueda acceder al pod del frontend en el puerto 5000. Sin restricción de origen para simular el NAT del firewall perimetral.

```
spec:
  podSelector:
    matchLabels:
      app: inventario-web
  policyTypes: [Ingress]
  ingress:
    - ports:
        - protocol: TCP
          port: 5000
```

5.2.3 allow-web-to-mongo

Solo el pod con label app: inventario-web puede iniciar conexiones al pod con label app: inventario-db en el puerto 27017. Cualquier otro pod que intente conectarse a MongoDB será bloqueado.

```
spec:
  podSelector:
    matchLabels:
      app: inventario-db
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: inventario-web
      ports:
        - protocol: TCP
          port: 27017
```

5.2.4 allow-dns

Permite que todos los pods realicen consultas DNS (UDP/TCP en el puerto 53) al servicio kube-dns del cluster, necesario para la resolución de nombres de servicio internos.

5.3 Configuración del Frontend (hostNetwork)

El pod del frontend usa hostNetwork: true para poder acceder al servidor SQL Server y al servidor LDAP que corren fuera del cluster (en el host o en otra máquina de laboratorio). La política DNS correspondiente es ClusterFirstWithHostNet.

5.4 Jobs de Inicialización de Bases de Datos

5.4.1 mongo-seed

El Job mongo-seed corre una vez y utiliza la imagen mongo:4.4 para conectarse al servicio inventario-db y ejecutar un script que inserta 10 documentos de hardware de ejemplo representando equipos reales de los laboratorios (Dell OptiPlex, HP EliteDesk, Lenovo ThinkCentre, workstations Precision y notebooks ThinkPad).

5.4.2 mssql-seed

El Job mssql-seed utiliza la imagen mssql-tools para ejecutar el script init.sql contra el servidor SQL Server. Este script crea la base de datos ubicacion_db, define las 5 tablas del modelo relacional, establece las restricciones de integridad referencial y carga datos de ejemplo (3 laboratorios, 3 responsables, 10 equipos, 2 asignaciones activas y 4 registros de mantenimiento).

5.5 Guía de Despliegue Paso a Paso

```
# 1. Iniciar Minikube con Calico:
minikube start --cni=calico

# 2. Construir la imagen del frontend:
eval $(minikube docker-env)
docker build -t inventario-web:latest ./app

# 3. Aplicar recursos en orden:
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/secrets.yaml
kubectl apply -f k8s/frontend/configmap.yaml
kubectl apply -f k8s/mongo-db/pvc.yaml
kubectl apply -f k8s/mongo-db/deployment.yaml
kubectl apply -f k8s/sql-db/pvc.yaml
kubectl apply -f k8s/sql-db/deployment.yaml
kubectl apply -f k8s/frontend/deployment.yaml
kubectl apply -f k8s/network-policies/

# 4. Esperar que los pods estén Ready:
kubectl get pods -n inventario-seguro -w
```

```
# 5. Ejecutar los jobs de seed:
kubectl apply -f k8s/mongo-db/seed-job.yaml
kubectl apply -f k8s/sql-server/sql-seed-job.yaml

# 6. Obtener la URL de acceso:
minikube service inventario-web -n inventario-seguro --url
```


6. Seguridad del Ecosistema

6.1 Autenticación y Autorización

La seguridad de acceso se implementa en múltiples capas:

- Capa de red: NetworkPolicies Kubernetes bloquean todo tráfico no autorizado entre pods.
- Capa de aplicación: el decorador `@login_required` verifica la sesión Flask en cada ruta protegida.
- Capa de autorización: el decorador `@role_required(permission)` verifica que el rol del usuario tenga el permiso necesario para la ruta; en caso contrario devuelve HTTP 403.
- Capa de identidad: las credenciales se validan contra Active Directory. La sesión Flask almacena el perfil del usuario (username, nombre, rol, email).

6.2 Gestión de Secretos en Kubernetes

Las credenciales sensibles se almacenan en objetos Secret de Kubernetes (codificados en Base64) y se inyectan como variables de entorno en los contenedores. Nunca se incluyen en el código fuente ni en los ConfigMaps:

Secret	Claves que contiene
app-secret	MONGO_USER, MONGO_PASSWORD, SQL_SERVER_USER, SQL_SERVER_PASSWORD, LDAP_BIND_USER, LDAP_BIND_PASSWORD, SECRET_KEY
sql-secret	SA_PASSWORD (contraseña del usuario SA de SQL Server)

6.3 Simulación del Perímetro Firewall

El enunciado exige simular que el tráfico entrante pasa obligatoriamente por una IP autorizada de la DMZ. En la implementación con Minikube:

- El servicio del frontend expone un NodePort (30080) accesible solo desde la IP del nodo de Minikube.
- La IP del nodo de Minikube actúa como la IP de la DMZ del firewall.
- GFW en el sistema host puede configurarse para restringir el acceso al NodePort únicamente desde rangos IP autorizados, emulando las reglas de pfSense.
- Las NetworkPolicies internas aseguran que, incluso si alguien accede a la red del cluster, no pueda comunicarse con las bases de datos sin pasar por el frontend.

6.4 Principio de Menor Privilegio en Red

¿Puede comunicarse?	inventario-web	inventario-db	ubicacion-db	ldap-service	kube-dn
Tráfico externo (internet)	Sí (:5000)	No	No	No	No

¿Puede comunicarse?	inventario-web	inventario-db	ubicacion-db	ldap-service	kube-dn
inventario-web	—	Sí (:27017)	Sí (host net)	Sí (:389)	Sí (:53)
inventario-db	No	—	No	No	Sí (:53)
ubicacion-db	No	No	—	No	Sí (:53)
ldap-service	No	No	No	—	Sí (:53)

7. Estructura del Repositorio Git

7.1 Árbol de Directorios

```

ecosistema-inventario-seguro/
├── .env.example           # Variables de entorno de ejemplo
├── .gitignore
├── README.md             # Guía rápida de levantamiento
├── scripts/
│   └── start.sh          # Script de despliegue automatizado
├── app/                  # Código fuente Flask
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── app.py            # Rutas y lógica de negocio
│   ├── auth.py           # Autenticación y autorización
│   ├── config.py         # Configuración centralizada
│   ├── db.py             # Capa de acceso a datos
│   ├── test_ldap.py      # Test de conectividad LDAP
│   ├── test_sql.py       # Test de conectividad SQL
│   ├── static/
│   │   ├── css/styles.css
│   │   └── js/main.js
│   └── templates/        # Plantillas Jinja2
│       ├── base.html, layout.html
│       ├── login.html, dashboard.html
│       ├── equipo_detalle.html, equipo_form.html
│       ├── componentes.html, componente_form.html, componente_detalle.html
│       ├── asignaciones.html, solicitud_form.html, solicitar_equipo.html
│       ├── mantenimiento_list.html, mantenimiento_form.html
│       └── 403.html, stub.html
├── db/
│   ├── mongo/            # (Documentación de queries MongoDB)
│   └── sql/              # (Scripts SQL adicionales)
├── docs/                 # Documentación del proyecto
├── ldap/                 # Configuración LDAP
└── k8s/                  # Manifiestos Kubernetes
    ├── namespace.yaml
    ├── secrets.yaml
    ├── frontend/
    │   ├── configmap.yaml
    │   ├── deployment.yaml # Deployment + Service NodePort
    │   └── secret.yaml
    ├── mongo-db/
    │   ├── deployment.yaml # Deployment + Service ClusterIP
    │   ├── pvc.yaml
    │   └── seed-job.yaml    # Job de carga inicial
    ├── sql-db/
    │   ├── deployment.yaml # Deployment + Service ClusterIP
    │   └── pvc.yaml
    ├── sql-server/
    │   └── sql-seed-job.yaml # Job + ConfigMap con init.sql
    └── ldap/

```

```
└─ network-policies/
   └─ 01-deny-all.yaml
   └─ 02-allow-external-to-web.yaml
   └─ 03-allow-web-to-mongo.yaml
   └─ 04-allow-dns.yaml
```

7.2 Estrategia de Ramas

Rama	Responsable	Contenido
main	Kevin Nieto	Código integrado, estable y revisado. Rama de entrega.
feature/devops-k8s	Lorenzo Zamparini	Todos los manifiestos Kubernetes, Dockerfile, scripts de despliegue.
feature/sql-db	Máximo Muñoz	Schema SQL Server, script init.sql, seed job, documentación del modelo relacional.
feature/mongo-db	Juan Manuel Atencio	Documentos de seed MongoDB, queries de demostración, schema documentation.
feature/ldap-network	Francisco Navas	Configuración LDAP, auth.py, NetworkPolicies, documentación de seguridad.
feature/flask-app	Penoff Pablo	app.py, db.py, config.py, templates Jinja2, estilos CSS, JavaScript.

8. Entregables del Proyecto

#	Entregable	Estado	Descripción
1	Documentación técnica	✓ Completo	Este documento: arquitectura, BD, flujo, red, despliegue
2	Esquema de arquitectura	✓ Completo	Sección 2: servicios, puertos, reglas de red
3	Modelo de base de datos	✓ Completo	Sección 3: DDL SQL Server + schema MongoDB con ejemplos
4	Diagrama de flujo	✓ Completo	Sección 4.6: flujo completo de la aplicación
5	Repositorio Git versionado	✓ Completo	Ramas por integrante, historia de commits visible
6	Manifiestos Kubernetes	✓ Completo	k8s/: namespace, deployments, services, PVCs, jobs, NP
7	Script de seed SQL Server	✓ Completo	k8s/sql-server/sql-seed-job.yaml (ConfigMap + Job)
8	Script de seed MongoDB	✓ Completo	k8s/mongo-db/seed-job.yaml (10 documentos reales)
9	Código de la aplicación web	✓ Completo	app/: Flask, templates, estáticos, tests de conectividad
10	Ecosistema funcional en Minikube	✓ Funcional	Todos los servicios se despliegan con scripts/start.sh
11	Presentación PowerPoint	✓ Completo	Generado en formato .pptx compatible con el enunciado

9. Glosario y Referencias

9.1 Glosario de Términos

Término	Definición
Minikube	Herramienta que ejecuta un cluster Kubernetes de un nodo en una máquina local, usada para desarrollo y pruebas.
Calico	Plugin CNI (Container Network Interface) que implementa NetworkPolicies en Kubernetes con alto rendimiento.
CNI	Container Network Interface: estándar que define cómo los plugins de red configuran las interfaces de los contenedores.
NetworkPolicy	Recurso Kubernetes que define reglas de firewall a nivel de pod, controlando el tráfico Ingress (entrante) y Egress (saliente).
Zero-Trust	Modelo de seguridad que niega todo el acceso por defecto y solo permite conexiones explícitamente autorizadas.
hostNetwork	Configuración de pod que comparte el stack de red del nodo host, permitiendo acceso a servicios externos al cluster.
ClusterIP	Tipo de Service Kubernetes que expone el servicio solo dentro del cluster, sin acceso externo.
NodePort	Tipo de Service Kubernetes que expone el servicio en un puerto estático del nodo, accesible desde fuera del cluster.
PVC	PersistentVolumeClaim: solicitud de almacenamiento persistente en Kubernetes, sobrevive al reinicio de pods.
LDAP	Lightweight Directory Access Protocol: protocolo para acceder a directorios de información de usuarios.
Active Directory	Implementación de LDAP de Microsoft para gestión centralizada de usuarios, grupos y políticas en redes Windows.
sAMAccountName	Atributo de Active Directory que contiene el nombre de usuario de red (login) del usuario.
pymssql	Driver Python para conectarse a Microsoft SQL Server.
pymongo	Driver oficial de Python para MongoDB.
Jinja2	Motor de plantillas Python usado por Flask para renderizar HTML dinámico en el servidor.
DMZ	Zona Desmilitarizada: segmento de red intermedio entre internet y la red interna, donde se exponen servicios públicos.
pfSense	Distribución de firewall/router de código abierto basada en FreeBSD.
GFW	Graphical Uncomplicated Firewall: interfaz gráfica para UFW, el firewall de Linux.

9.2 Referencias Técnicas

- Kubernetes NetworkPolicies: <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- Calico CNI: <https://docs.projectcalico.org/>
- Flask Documentation: <https://flask.palletsprojects.com/>
- MongoDB Manual: <https://www.mongodb.com/docs/manual/>
- SQL Server 2022: <https://learn.microsoft.com/en-us/sql/sql-server/>
- ldap3 Python: <https://ldap3.readthedocs.io/>
- Minikube: <https://minikube.sigs.k8s.io/docs/>